Project: Roots of Equations

1- Objective

The aim of this project is to compare and analyze the behavior of the numerical methods used for calculating the roots of equations, on a simulated machine that keeps only *n* significant figures:

1. Bisection.
2. False position.
3. Fixed point.
4. Original Newton–Raphson.
5. Modified Newton–Raphson.
6. Secant.

2- Description

You are required to implement `numcomp`, a Python package that takes an equation, the method to use and its parameters, and returns the approximate root together with the full table of iterations. Every arithmetic operation is rounded (or chopped) to *n* significant figures, exactly like the hand calculations in the tutorials, so round-off error, subtractive cancellation, stagnation and cycling can be observed. There is no GUI: the package has a fixed API that is checked by automatic tests.

The complete specification (function signatures, the exact order of operations, stopping rules, results and errors) is in `README.md` of the starter repository, `github.com/ZeyadZakaria01/numcomp-starter`. Where this document and the README differ, the README is authoritative.

3- Specification

You are required to complete three files of the starter code:

1. `machine.py`: the simulated machine.
   a. `round_sig(value, n, mode)` rounds a number to *n* significant figures, by rounding (half away from zero) or chopping.
   b. `Machine(sig_figs, mode)` provides addition, subtraction, multiplication, division, integer powers and elementary functions. Each operation is computed exactly and then rounded once. `sig_figs = None` means ordinary double precision.
2. `roots.py`: the six methods. Each method accepts:
   a. A free-text equation in `x`, using `+ - * / ^` and `exp`, `sin`, `cos`, `tan`, `ln`, `sqrt`, `abs` (parsed by the provided `expr.py`).
   b. The starting values and options listed in the table below. Default values: #SFs = double precision, EPS = $0.5 \times 10^{2-n}$ % (0.001 % in double precision), max iterations = 50.

   Each method returns the approximate root, number of iterations, approximate relative error, number of correct significant figures, the residual, the reason it stopped, the execution time and the table of

iterations. Error messages (no bracket, zero derivative, divergence, invalid expression, …) must be descriptive.

3. `plot.py`: plot *f(x)*, or *g(x)* with *y* = *x* in case of fixed point, to help choose the initial guess(es).

Method	Input	Parameters
Bisection	*f(x)*	Interval `xl`, `xu`
False position	*f(x)*	Interval `xl`, `xu`
Fixed point	*g(x)*, from *x* = *g(x)*	Initial guess `x0`
Newton–Raphson	*f(x)*	Initial guess `x0` (*f′* is computed symbolically)
Modified Newton–Raphson	*f(x)*	Initial guess `x0` (*f′* and *f″* are computed symbolically)
Secant	*f(x)*	Initial guesses `x0`, `x1`
All methods: #SFs (`sig_figs`), rounding or chopping (`mode`), EPS (`eps`), max iterations (`max_iter`) and, optionally, the true root (`true_root`) to report the true number of correct significant figures.		

4- Deliverables

You must deliver the following:

1. The code: your team's private repository, made from the starter with "Use this template" (not a fork), clean and commented. Every member commits under their own account.
2. A well-formatted report that contains the following:
   a. Pseudo-code or flowchart for each method.
   b. The plots used to choose the starting values.
   c. Sample runs for each method, covering normal and tricky cases (no bracket, stagnation, cycling, divergence, double root) and the effect of *n* and of rounding versus chopping.
   d. Analysis of the performance of all the methods on at least two different equations, comparing the number of iterations, the final approximate relative error, the number of correct significant figures and the execution time.

5- Grading

1. Public tests (`uv run pytest`) run on every push; private tests use the same test code on many more cases. Passing the public tests is necessary, not sufficient.
2. Individual discussion: predict a few rows of a table by hand, explain a failure (cancellation, stagnation, one-sided false position, …), and make a small change to your code on the spot.

6- Notes

Please note the following:

1. Allowed: the Python standard library, SymPy through the provided `expr.py` only, and matplotlib / NumPy in `plot.py` only. Not allowed: `decimal`, the built-in `round()`, `scipy`, `mpmath`, `eval` / `exec` and SymPy's solvers.
2. Do not edit the provided files; the grader replaces them with the originals.
3. AI tools must not be used to write the code or the report; this, and copying from other teams or previous years, falls under the cheating policy. Code you cannot explain and modify yourself in the discussion counts as not yours.